

Global Project Management Tool

GPM

System Design Package — v2.0

Prepared for: Development Team

Purpose: Complete functional and technical specification for the GPM platform build

Magdynamics · July 2026

Contents

System Design Package — v2.0

1. Executive Summary

- Design principles
- The five layers

2. System Architecture

- 2.1 The event log is the keystone
- 2.2 Uniform write path

3. Core Data Model

- 3.1 Identity & org
- 3.2 Catalog & work
- 3.3 PMO
- 3.4 Workspace & knowledge
- 3.5 Agent runtime

4. Admin Platform Layer

- 4.1 User management
- 4.2 Department & team management
- 4.3 Dashboards (derived, never hand-set)
- 4.4 Knowledge base

5. Project Catalog

6. Program Management Office (PMO)

- 6.1 Requirements registry
- 6.2 Structured acceptance criteria
- 6.3 Flags — the lifecycle that makes enforcement real
- 6.4 Derived statuses
- 6.5 Approval gates
- 6.6 Audit trail — "immutable," concretely

7. UX Governance — enforced by pipeline, not committee

- 7.1 The handbook is the component library
- 7.2 Interaction & accessibility specs
- 7.3 The compliance gate (automated)
- 7.4 The one human gate

8. Workspace Layer

- 8.1 Idea intake

- 8.2 Three-way decision
- 8.3 Project conversion (on Go)
- 8.4 Scheduling
- 8.5 Launch meeting — capture, then ratify
- 8.6 Communication — bind Slack, don't rebuild it

9. AI Agent Ecosystem — six services, one runtime

- 9.1 The six services
- 9.2 Agent governance rules (binding)
- 9.3 Measurement & promotion
- 9.4 Model assignment

10. Integrations

11. Notifications

12. Authentication & security

13. Implementation Roadmap

14. Definition of "Fully Operational"

- 14.1 Functional
- 14.2 Operational
- 14.3 Agent quality
- 14.4 Adoption — the real bar

15. Resolved Decisions (formerly "Open Questions")

Deliberate non-goals (scope traps, declined)

System Design Package — v2.0

Prepared for: Development Team **Purpose:** Complete functional and technical specification for the GPM platform build **Supersedes:** v1.0. All review findings from `GPM-design-review-and-build-guide.md` are incorporated; that document remains as the rationale record for the changes.

Changelog from v1.0 (summary):

- Added the missing `Task` entity and full core data model (§3)
- Added first-class `Flag` entity with lifecycle, ownership, and escalation (§6.3)
- Added event-log architecture: every agent and rule is an event subscriber (§2, §10)
- Consolidated 15 agents into 6 runtime services with a governance model (§9)
- Replaced manual UX change-control with CI-enforced governance (§7)
- Requirement/project statuses are now derived, never hand-set (§6.4)
- Acceptance criteria are structured checklists, not prose (§6.2)
- Communication: Slack-bound channels + distillation, not a built-in chat system (§8.6)
- Tabled ideas get mandatory revisit triggers (§8.2)
- Agents are first-class principals in RBAC and the audit log (§4.1, §9.2)
- Roadmap re-sequenced: the PMO enforcement core ships in Phase 1 (§13)
- The four open questions are resolved as decisions (§15)

1. Executive Summary

GPM is an internal platform giving a small, multi-hat team (3–5 people spanning business strategy, IT operations, development, and architectural/structural design) centralized visibility and control over projects across multiple business units.

The one problem GPM exists to solve: agreed requirements silently going undone. Every scope decision is tested against it.

Design principles

1. **Minimize manual entry, maximize automated capture.** A governance tool that demands data entry from five busy people dies in a month.
2. **Agents propose; rules enforce.** Anything that blocks or unblocks work — gates, stage transitions, flag rules — is deterministic code. LLM output feeds human decisions or raises flags; it never silently gates.

3. **Nothing flags into a void.** Every flag has an owner, a lifecycle, and an escalation path. An alert without an owner is a requirement silently going undone, one level up.
4. **Capture where people already are.** Intake from Slack/email; discussion in project-bound Slack channels; the platform stores the distilled record.
5. **Draft first, auto later.** Every agent capability launches in draft mode (human ratifies its output) and is promoted to auto-commit only on measured precision.
6. **Derived over declared.** Statuses roll up from underlying data; they cannot be hand-set into optimistic fiction.

The five layers

1. **Admin Platform** — users (human *and* agent principals), departments, teams, capacity, dashboards, knowledge base
2. **Program Management Office (PMO)** — requirements, flags, approval gates, compliance, audit trail: the enforcement backbone
3. **UX Governance** — a design system enforced by CI pipeline, not by committee
4. **Workspace** — intake → decision → conversion → scheduling → launch → distilled communication
5. **AI Agent Ecosystem** — six runtime services (consolidating fifteen v1.0 agent roles) subscribing to the platform event log

2. System Architecture

GPM is a **modular monolith around an event log** — not five stacked services. One web app, one API service, one worker, one Postgres database.



2.1 The event log is the keystone

Every mutation emits a domain event into an append-only outbox table:

```
idea.received · intake.structured · decision.made · project.created ·
charter.ratified · task.created · task.completed · requirement.created ·
requirement.updated · requirement.attested · gate.approved · gate.blocked ·
stage.advanced · flag.raised · flag.resolved · flag.escalated ·
content.ingested · asset.linked · change.requested · agent.run_completed
```

Rule packs and agent services are **subscribers**. This one pattern answers "when do agents run," powers the audit trail, enables replay/debugging, and keeps humans and agents on the same write path.

2.2 Uniform write path

Agents call the same API layer as humans, authenticated as agent principals with scoped permissions — never raw SQL. RBAC and audit therefore apply uniformly; "who did this and why" has one answer format for everyone.

3. Core Data Model

Entities marked ★ were missing or underspecified in v1.0.

3.1 Identity & org

```
User:          id, name, email, kind (human | agent ★), status, sso_subject
Role:          Admin | DepartmentLead | TeamLead | Contributor | AgentScoped ★
Department:    id, name, sponsor_id (FK → User ★, not free text)
Team:          id, name, department_id, charter_text
TeamMembership: team_id, user_id, allocation_fraction
Skill ★:       id, tag -- flat list, ≤30 tags; no framework
MemberSkill ★: user_id, skill_id, level (1-3)
Commitment ★:  id, user_id, project_id, hours_per_week, start_date, end_date
```

Capacity and utilization are **queries** over **TeamMembership** + **Commitment** (hours_per_week × allocation minus committed hours) — not a separate tracked dataset.

3.2 Catalog & work

```

Project:      id (system-generated), name, description, department_id,
              sponsor_id, team_id, stage, idea_id * (lineage), charter_id *,
              created_at
Task *:       id, project_id, title, description, owner_id (human or agent),
              status (todo | in_progress | blocked | done | cancelled),
              due_date, estimate_hours, created_by, source (manual|agent|import),
              external_ref (e.g. GitHub issue URL)
TaskDependency *: task_id, depends_on_task_id
RequirementTask *: requirement_id, task_id          -- the mapping flag rules run on
AssetLink *:   id, project_id, type (github|drive|other), url, external_id,
              verified_at                          -- verified by reconciliation job

```

Stage enum: Planning / Design / Development / Testing / Deployment / Maintenance. **Transition rule (mechanical, not advisory):** stage advances only when every gate configured for the current stage is approved. This is the entire link between the catalog and the PMO, and it is enforced in code.

Naming convention: every linked asset follows `GPM-{project_id}-{short-name}` so the nightly reconciliation job can auto-discover, match, and flag orphans.

3.3 PMO

```

Requirement:   id, project_id, type (functional | non_functional | gate),
              owner_id, due_date, status (derived – see §6.4), created_at
RequirementVersion: requirement_id, version, body, changed_by, changed_at
AcceptanceCriterion *: id, requirement_id, text,
                      check_mode (human_attest | rule | agent_assessed),
                      satisfied_by (attestation ref), satisfied_at
ApprovalGate: id, project_id, stage, name,
              kind (human | automated *), config
GateApproval: gate_id, approver_id (human or agent principal),
              evidence_ref (e.g. CI run *), approved_at  -- append-only
Flag *:       id, type (unlinked_requirement | scope_slip | overdue |
                      dependency_risk | gate_blocked | capacity | drift | custom),
              severity (info | warn | high | critical),
              subject_type, subject_id, raised_by (rule id or agent id),
              raised_at, status (open | acknowledged | resolved | dismissed),
              owner_id, resolution_note, resolved_at
AuditLogEntry: actor_id, action, subject_type, subject_id,
              before (JSON), after (JSON), event_id, at  -- append-only
ChangeRequest *: id, project_id, description, impact_analysis (JSON),
                status, decided_by, decided_at

```

3.4 Workspace & knowledge

```

Idea:                id, raw_content, source (form|slack|email), submitted_by, at
IntakeRecord:        id, idea_id, problem_statement, stakeholders[],
                    effort_estimate, dependencies[], duplicates_of *
Decision:            id, intake_id, outcome (go | no_go | table),
                    reasoning (required), decided_by, decided_at,
                    revisit_on * (date) / revisit_when * (condition) -- required if tabled
LaunchMeetingRecord: id, project_id, transcript_ref, ratified_at
RatificationItem *:  id, source_run_id, kind (requirement|task|decision|knowledge),
                    payload (JSON), status (pending|accepted|edited|rejected),
                    reviewed_by, reviewed_at
KnowledgeArticle:    id, title, body, tags[], version,
                    provenance_ref * (event/run that generated it)

```

3.5 Agent runtime

```

AgentCapability *:   id, service, name, trigger_events[], output_schema_ref,
                    write_scope[], mode (draft | auto), token_budget
AgentRun *:          id, capability_id, trigger_event_id, input_refs[],
                    output (JSON), status, tokens, cost, latency_ms, at
Event (outbox):      id, type, subject, payload (JSON), at      -- append-only

```

4. Admin Platform Layer

4.1 User management

- RBAC: **Admin, Department Lead, Team Lead, Contributor** — plus **agent principals**: every AI agent service is a **User** with **kind = 'agent'** and narrowly scoped permissions (**write_scope** per capability). Agents appear in the audit log exactly like humans.
- Provisioning/deactivation via Google Workspace SSO (§12); deactivating the Google account deactivates the GPM user.
- Audit log of all permission changes (append-only, §6.6).

4.2 Department & team management

- CRUD for departments (sponsor is a **User** FK) and teams; team profile carries roster, allocation fractions, skill tags, and charter text.
- **Deferred to Phase 5**: historical team performance (velocity, on-time rate). These are derived from task-completion data and are only meaningful after ~2 quarters of real usage — no separate tracking feature is built.

4.3 Dashboards (derived, never hand-set)

- **Cross-project status:** *at-risk* = any linked requirement At Risk/Overdue or any open flag $\geq$ high; *blocked* = a failing gate or open dependency flag; *on-track* otherwise. The derivation rules are code, versioned with the app.
- **Capacity vs. commitment:** per team/member, from `Commitment` queries.
- **Timeline view:** calendar-style from task due dates and stage gates (Gantt is a later nicety, not Phase 1).

4.4 Knowledge base

- Searchable, tagged by project/team/topic; version history per article.
- Auto-populated by the Extraction service (§9) **through the ratification queue** — auto-generated articles carry a `provenance_ref` to the discussion, meeting, or commit they came from, so readers can judge trust.

5. Project Catalog

Every project is the organizing hub; all other data hangs off its ID (schema in §3.2). Key behaviors:

- `project_id` auto-assigned on intake approval (Go decision → Provisioning service).
- Lineage is walkable both directions: `Project.idea_id` → `Decision` → `IntakeRecord` → `Idea`, and forward into requirements, tasks, and attestations.
- Asset links are **reconciled nightly**: the worker scans GitHub/Drive for `GPM-*` names, matches to projects, sets `verified_at`, and raises `drift` flags for orphans or misnamed assets. (Deterministic string matching; LLM only proposes fuzzy matches, as ratification items.)

6. Program Management Office (PMO)

The enforcement backbone. Ships **first** (Phase 1) because it is the named problem — and its core works with zero AI.

6.1 Requirements registry

Every commitment — functional, non-functional, or gate — is logged with a unique ID, type, owner, due date, structured acceptance criteria, and full version history (changes preserve the old version; see §3.3).

6.2 Structured acceptance criteria

Criteria are a **checklist of individually checkable assertions**, not a prose blob. Each criterion declares its `check_mode`:

- `human_attest` — a named human marks it satisfied (recorded as attestation);
- `rule` — a deterministic check (e.g., CI green, file exists at path);
- `agent_assessed` — the Verification service assesses and **proposes**; a human confirms before it counts.

A requirement is Complete only when **all** criteria are satisfied. This checklist is what makes agent accountability (§9.2) mechanical instead of aspirational.

6.3 Flags — the lifecycle that makes enforcement real

All detections land as `Flag` rows (schema §3.3), never as fire-and-forget notifications. Rules:

1. **Every flag has an owner** (defaulting to the subject's owner, else the team lead).
2. Dismissing a flag **requires a resolution note**.
3. `critical` flags open longer than **3 days** auto-escalate to the department sponsor (`flag.escalated` event → sponsor digest + Slack ping).
4. Flag volume and dismissal rate are tracked per raising rule/capability; a rule whose flags are dismissed >30–40% of the time is tuned or retired — false alarms destroy trust in the backbone.

Core deterministic flag rules (SQL/code, run on relevant events + nightly sweep):

- requirement with zero linked tasks → `unlinked_requirement`;
- all linked tasks done but requirement not complete → `scope_slip` (review);
- task/requirement past due, not done → `overdue`;
- task blocked by an incomplete dependency past its due date → `dependency_risk`;
- member committed > available hours → `capacity`;
- asset naming/reconciliation mismatch → `drift`.

6.4 Derived statuses

Requirement status is computed, never typed:

- **Not Started** — no linked task started
- **On Track** — tasks progressing, due dates safe, no open flag ≥ high
- **At Risk** — open high-severity flag or forecast slip
- **Overdue** — past due, not complete
- **Complete** — all acceptance criteria satisfied *and* attested

Compliance dashboard rolls these up to project- and portfolio-level compliance %.

6.5 Approval gates

- Configurable per project and stage (e.g., architectural sign-off before Development, stakeholder approval before Deployment, UX pipeline green before ship).
- Gates **mechanically block** stage progression (§3.2 transition rule).
- Two kinds: **human** (approval from an authenticated session of a named person — no approving on someone's behalf) and **automated** (a named agent principal records the approval with evidence attached, e.g. the CI run — see §7.3).
- Every approval is append-only: who, when, what, evidence.

6.6 Audit trail — "immutable," concretely

- Append-only **audit_log** table: the application DB role has **no UPDATE/DELETE grants**; writes go through a single audited code path; every entry carries actor, action, before/after JSON, and the triggering **event_id**.
- Agent actions log the agent principal and the trigger. Nightly off-site backups make the log durable; hash-chaining is unnecessary for an internal tool.
- Exportable compliance reports: committed vs. delivered, by project or team.

7. UX Governance — enforced by pipeline, not committee

Goal unchanged from v1.0: consistent, high-quality UI/UX as a system requirement. Mechanism changed: **the handbook is code, and compliance is CI**. A manual change- request gate on every UI change would be rubber-stamped or bypassed by a 3–5 person team; a pipeline cannot be.

7.1 The handbook is the component library

- Design tokens (color, spacing, type) in a token file; components in a shared library.
- **Storybook is the living handbook**: every component with its states (default, hover, error, disabled, loading), plus per-screen user stories (who uses it, what problem it solves, expected flow) as markdown alongside the components.
- Design-system versions = git tags of the library. Version history is free.

7.2 Interaction & accessibility specs

- Click behavior, validation, error and loading states written as **Playwright tests**.
- Accessibility: **axe-core** in CI, keyboard-nav smoke tests, contrast checked at the token level (tokens can't merge if a pairing fails contrast).

7.3 The compliance gate (automated)

Before a screen ships, CI verifies:

1. screens import only from the component library (lint rule — no ad-hoc styles);
2. visual regression (screenshot diff) against approved baselines;
3. interaction tests pass;
4. accessibility checks pass.

A green pipeline is the §6.5 gate approval, recorded by the `UX-CI` agent principal with the run attached. No screen bypasses governance — because governance runs on every commit.

7.4 The one human gate

Human review is reserved for **changes to the handbook itself**: a new pattern, a changed token or component. That is rare enough to review properly, and it routes through a PMO change request like any scope change (§9.1, service 6).

8. Workspace Layer

Ideas become projects here. Agents do the structuring; humans make the judgment calls.

8.1 Idea intake

- Ideas arrive via form, Slack, or email. The Intake capability structures them: problem statement, stakeholders, rough effort, dependencies — no form-filling.
- **Dedup**: intake checks similarity against open intakes/projects and proposes merges (`duplicates_of`) as ratification items.
- Output: a structured `IntakeRecord` awaiting decision.

8.2 Three-way decision

Every idea resolves to exactly one outcome, with reasoning logged in all cases:

- **Go** — proceeds to conversion (§8.3).
- **No-Go** — declined; reasoning preserved.
- **Table** — **requires a revisit trigger**: a date (`revisit_on`) or condition (`revisit_when`, e.g. "when Team A frees up"). A scheduled job re-surfaces tabled ideas when triggers fire. A graveyard of forgotten good ideas is the intake-stage version of the problem GPM exists to solve.

The Decision-Support capability produces a **scorecard** — capacity fit, skill fit, strategic fit, risk notes, each with the evidence used — not a bare recommendation. The team decides; the agent makes trade-offs visible.

8.3 Project conversion (on Go)

- Provisioning service creates the catalog entry, assigns `project_id`, links lineage, drafts the charter (team members, timeline, success criteria from project type and historical data) — **as a draft for ratification**, never auto-binding.

8.4 Scheduling

- Provisioning service generates the pre-launch checklist, books the launch meeting (Google Calendar), and flags resource conflicts against `Commitment` data.

8.5 Launch meeting — capture, then ratify

- Meeting transcript or shared notes → Extraction service proposes charter entries: decisions, assignments, success criteria, requirements, dependencies.
- Humans confirm each item** in the ratification queue before it enters the Requirements Registry. Auto-capture, human-ratify: unratified extraction never becomes a binding requirement.

8.6 Communication — bind Slack, don't rebuild it

- GPM does not include a chat system.** Each idea/project gets a bound Slack channel (or thread); people keep talking where they already talk.
- The Extraction service ingests bound channels: summaries, decisions, action items → ratification queue → PMO log and knowledge base.
- The workspace UI shows the **distilled record** — decisions, action items, summaries, linked to their source messages. That distilled record, not the raw chatter, is what governance needs.

9. AI Agent Ecosystem — six services, one runtime

The fifteen agent roles from v1.0 are consolidated by **runtime shape** into six services. Every v1.0 responsibility survives as a *capability* within a service; each capability is specced (trigger, inputs, output schema, write scope, mode, eval set, token budget) before it is built.

9.1 The six services

#	Service	Absorbs v1.0 agents	Shape
1	Intake & Decision	Intake, Decision-Support	Event: <code>idea.received</code> → structure, dedup, scorecard → await human decision
2	Provisioning	Project Charter, Scheduling, Archiving	Event: Go <code>decision.made</code> → catalog entry, charter draft, checklist, calendar booking; nightly asset reconciliation
3	Extraction	Communication, Knowledge Extraction	Event: <code>content.ingested</code> → summaries, decisions, action items, knowledge → ratification queue
4	Watchdog	Follow-up, Dependency, Capacity, Risk	Scheduled sweeps + events → raise/update/escalate Flags; mostly deterministic rules, LLM only for narrative risk summaries
5	Verification	Task Accountability, Requirements Traceability, Compliance Enforcement	Event: task/requirement marked done → check against acceptance criteria and UX pipeline → attest (propose) or flag <code>scope_slip/drift</code>
6	Change Management	Change Management	Event: <code>change.requested</code> → impact analysis over linked requirements/tasks/dependencies → route through gates

9.2 Agent governance rules (binding)

1. **Draft or auto, nothing in between.** Every capability launches in `draft` mode — humans ratify output. Promotion to `auto` (logged, reversible) requires the promotion bar in §9.3. Externally visible or destructive actions (sending messages, closing requirements) stay draft indefinitely.
2. **Agents propose; rules enforce.** Gates, stage transitions, and flag rules are deterministic code. LLM output never silently blocks or unblocks work.
3. **Structured outputs only.** Every LLM call returns JSON validated against the capability's schema before any write. Prose goes only in prose fields.
4. **Uniform write path.** Agents write via the API as scoped principals (§2.2); every run is an `AgentRun` row — trigger, inputs, output, tokens, cost, latency. This is v1.0 §5.5's "AI agent accountability," made mechanical.
5. **Degrade gracefully.** If the LLM API is down, the deterministic core (flags, gates, audit, dashboards) keeps working and the system falls back to manual entry. Agent outage is an inconvenience, never an enforcement outage.

9.3 Measurement & promotion

Per capability, tracked continuously and reviewed weekly (§14.3):

- proposals made / accepted / edited / rejected — **acceptance rate $\geq 80\%$** before promotion to auto;
- flag precision — dismissal rate **$< 30\%$** or the rule gets tuned/retired;
- token cost per capability per month, against its budget.

9.4 Model assignment

- `claude-sonnet-5` — default for structuring, extraction, verification;
- `claude-opus-5` — decision scorecards and change-impact analysis (high stakes, low volume);
- `claude-haiku-4-5` — high-volume summarization of bound channels.

10. Integrations

Priority order and purpose:

1. **Slack** — intake capture, flag/digest notifications, bound-channel ingestion. Powers three subsystems; first in.
2. **GitHub** — asset linking, task `external_ref` sync, CI signals for automated gates (including UX-CI).
3. **Google Calendar** — launch-meeting booking (Provisioning service).
4. **Google Drive** — asset reconciliation against the naming convention.
5. **Email** — intake only; last.

All webhooks validated; integration tokens stored as secrets with expiry monitoring (a dead token raises a `critical` flag — the backbone must not go blind silently).

11. Notifications

- The **in-app inbox is the system of record**: it is exactly the open-Flags view plus the ratification queue. Slack is a delivery channel, not the record.
- Defaults: per-person **daily digest**; real-time Slack ping only for `critical` flags and gate blocks; weekly email digest to sponsors. Per-user overrides.
- Design target: zero notification fatigue. A muted channel means the enforcement backbone goes blind — fewer, better alerts beat comprehensive noise.

12. Authentication & security

- **SSO via Google Workspace (OIDC)**. No standalone passwords; deactivation = disabling the Google account. The team already lives in Drive/Gmail.

- Agents authenticate as service principals with scoped API tokens (§4.1).
- Append-only audit and event tables enforced at the DB-grant level (§6.6).
- Nightly `pg_dump` off-site; restore tested (§14.2).

13. Implementation Roadmap

Re-sequenced from v1.0 so the **named problem is addressed in Phase 1** and every phase ends with something the team uses daily. Durations assume ~2 people part-time.

Phase 0 — Walking skeleton (1–2 weeks) Monorepo, CI, deploy pipeline, OIDC login, Postgres + migrations, event-outbox pattern proven end-to-end with one event, component library seeded (tokens + 5 base components + Storybook + axe in CI). *Exit: a deployed authenticated page; a commit violating a lint/a11y rule fails CI.*

Phase 1 — Enforcement core (3–4 weeks) Minimal admin CRUD, Project Catalog + ID generation, Tasks, Requirements Registry with versioning and structured acceptance criteria, requirement ↔ task mapping, **deterministic flag rules + Flag lifecycle UI**, append-only audit log, derived-status dashboard. *Exit: one real project run through it; both core flag rules fire on real data; every change audited. GPM is solving its named problem — with zero AI.*

Phase 2 — Gates & intake (3–4 weeks) Approval gates + mechanical stage blocking, intake (manual form first), three-way decision with logged reasoning and tabled-revisit triggers, project conversion, compliance roll-ups, Slack notifications (digest + critical pings). *Exit: a project cannot skip a gate; a tabled idea resurfaces automatically; sponsors get a weekly digest.*

Phase 3 — First agents, draft mode (4–6 weeks) Agent runtime (Claude Agent SDK, `agent_runs`, ratification queue). Ship by value-per-risk: ① Intake structuring, ② Extraction (bound channels + meeting notes), ③ Provisioning (charter draft, checklist, calendar), ④ Decision scorecard. *Exit: raw idea → structured intake with no form-filling; meeting → ratified requirements; every agent action visible with accept/edit/reject tracked.*

Phase 4 — Watchdog & verification (4–6 weeks) Watchdog sweeps (deadline/dependency/capacity/risk → owned, escalating flags), Verification service (criteria checking, scope-drift detection), UX pipeline wired as an automated gate, GitHub + Drive reconciliation. Promote proven Phase-3 capabilities to auto on §9.3 data. *Exit: a requirement cannot be Complete without attested criteria; missed deadlines escalate to sponsors without human triage.*

Phase 5 — Intelligence & polish (ongoing) Knowledge extraction with provenance, Change Management service with impact analysis, capacity forecasting, portfolio compliance exports, historical team-performance views (real data now exists), Gantt timeline.

14. Definition of "Fully Operational"

These are GPM's own PMO requirements: **load them into GPM in Phase 1 and let the system track its own completion.**

14.1 Functional

- Idea → decision → project → launch runs without manual data re-entry.
- All core flag rules run event-triggered plus nightly; flags carry owners and escalate per §6.3.
- Gates mechanically block stage transitions.
- Every requirement traceable end-to-end: idea → decision → charter → tasks → attestations.
- Audit export answers "committed vs. delivered" per project and team.

14.2 Operational

- Nightly off-site backups with a **tested** restore.
- One-command deploy with rollback.
- Failed agent runs alert and are replayable from the event log.
- Integration token expiry monitored (dead token = critical flag).
- Runbooks: Slack outage, LLM API outage (system degrades to manual mode — deterministic core keeps working), DB restore.

14.3 Agent quality

- Every capability has an eval set and live precision metrics (§9.3).
- ≥80% acceptance before any capability leaves draft mode; flag dismissal <30%.
- Monthly token cost visible per capability.
- **Weekly 30-minute governance review:** open flags, precision numbers, draft → auto promotions. This meeting is the human half of the PMO.

14.4 Adoption — the real bar

- The team runs **all** projects in GPM, including GPM's own build, for a full quarter.
- Zero governance work in side spreadsheets.
- At least one real "would have silently slipped" catch credited to the system in retrospective.

15. Resolved Decisions (formerly "Open Questions")

1. **Time tracking:** No actual-hours tracking in v1 — highest-friction data entry, and the named problem doesn't need it. Progress derives from task status + due dates; capacity from planned commitments (§3.1). Revisit only if forecasting proves inaccurate after two quarters.
2. **Integration priority:** Slack → GitHub → Google Calendar → Google Drive → email (§10).
3. **Notifications:** in-app inbox as system of record; Slack digests by default, real-time only for critical; weekly sponsor email (§11).
4. **Authentication:** Google Workspace SSO (OIDC); agents as scoped service principals; no standalone accounts (§12).

Deliberate non-goals (scope traps, declined)

- **No built-in chat system** — Slack binding + distillation (§8.6).
- **No time tracking** in v1 (§15.1).
- **No per-agent microservices** — six services, one runtime (§9).
- **No manual UX change-approval for routine work** — CI is the gate (§7).

End of design package v2.0.